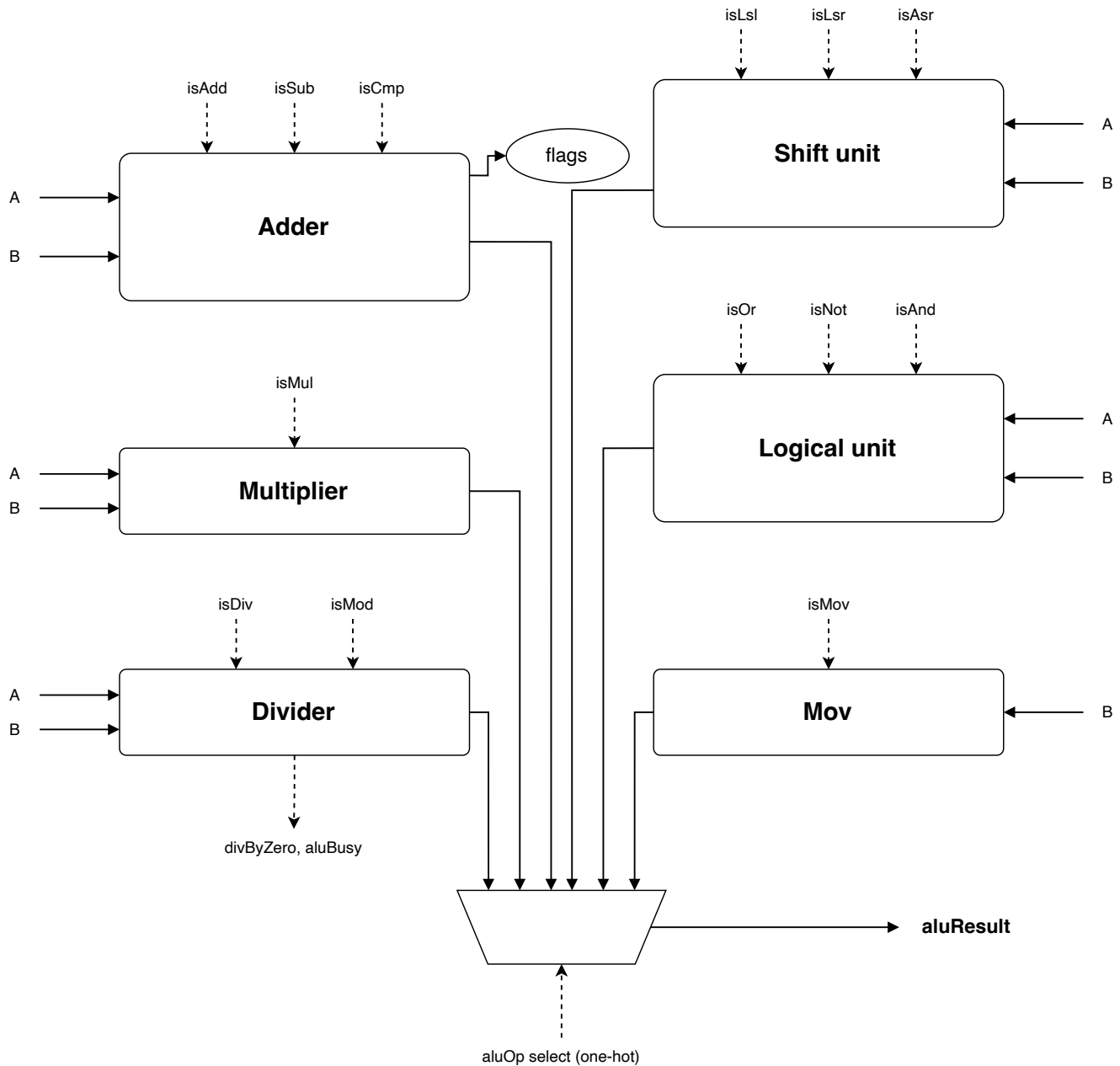


ALU

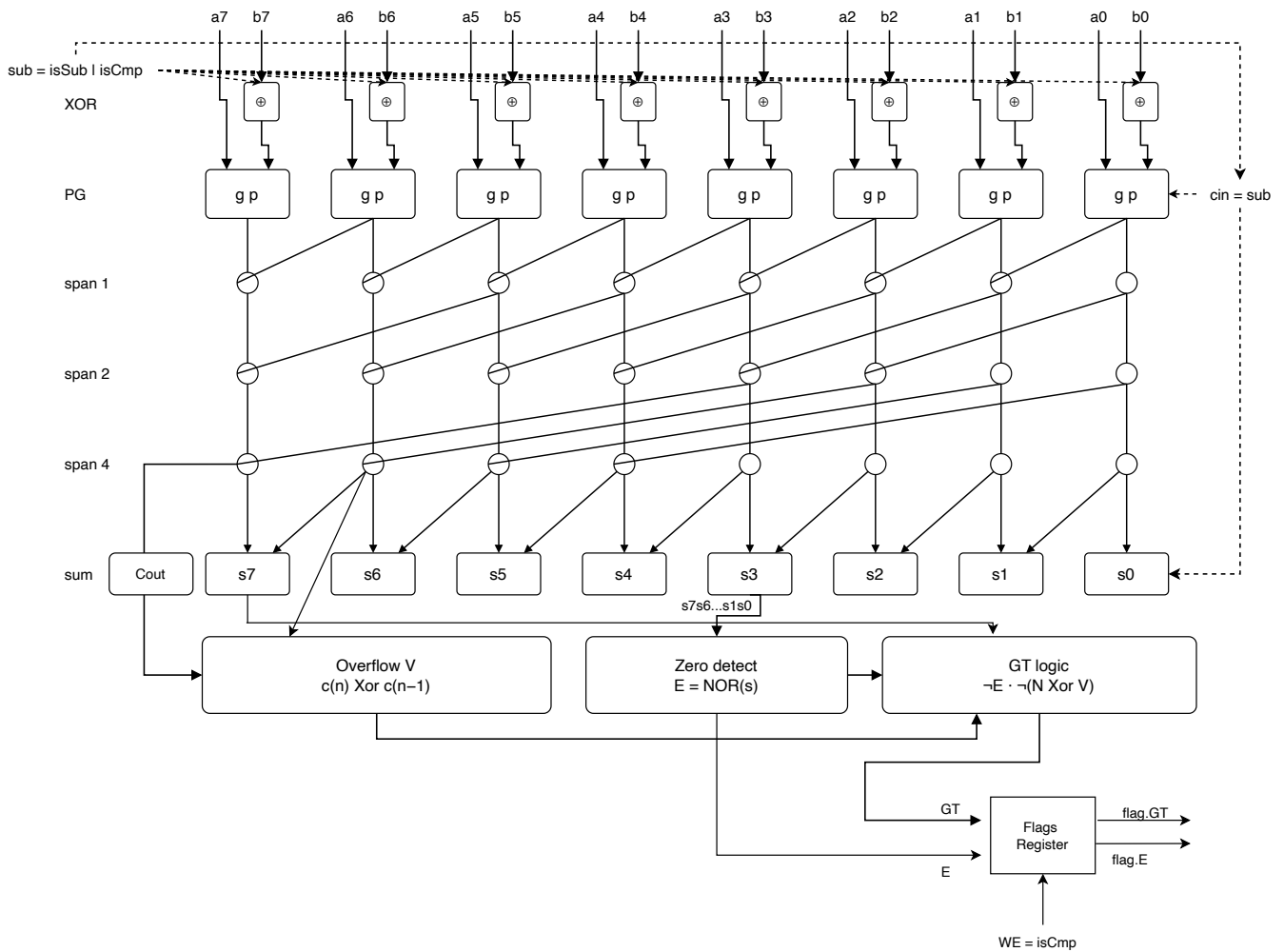
Overall Structure



Adder / Subtractor

Implemented using the Kogge-Stone parallel-prefix algorithm, as it gives the minimum carry delay (5 levels for 32 bits) with low fanout, at the cost of extra area.

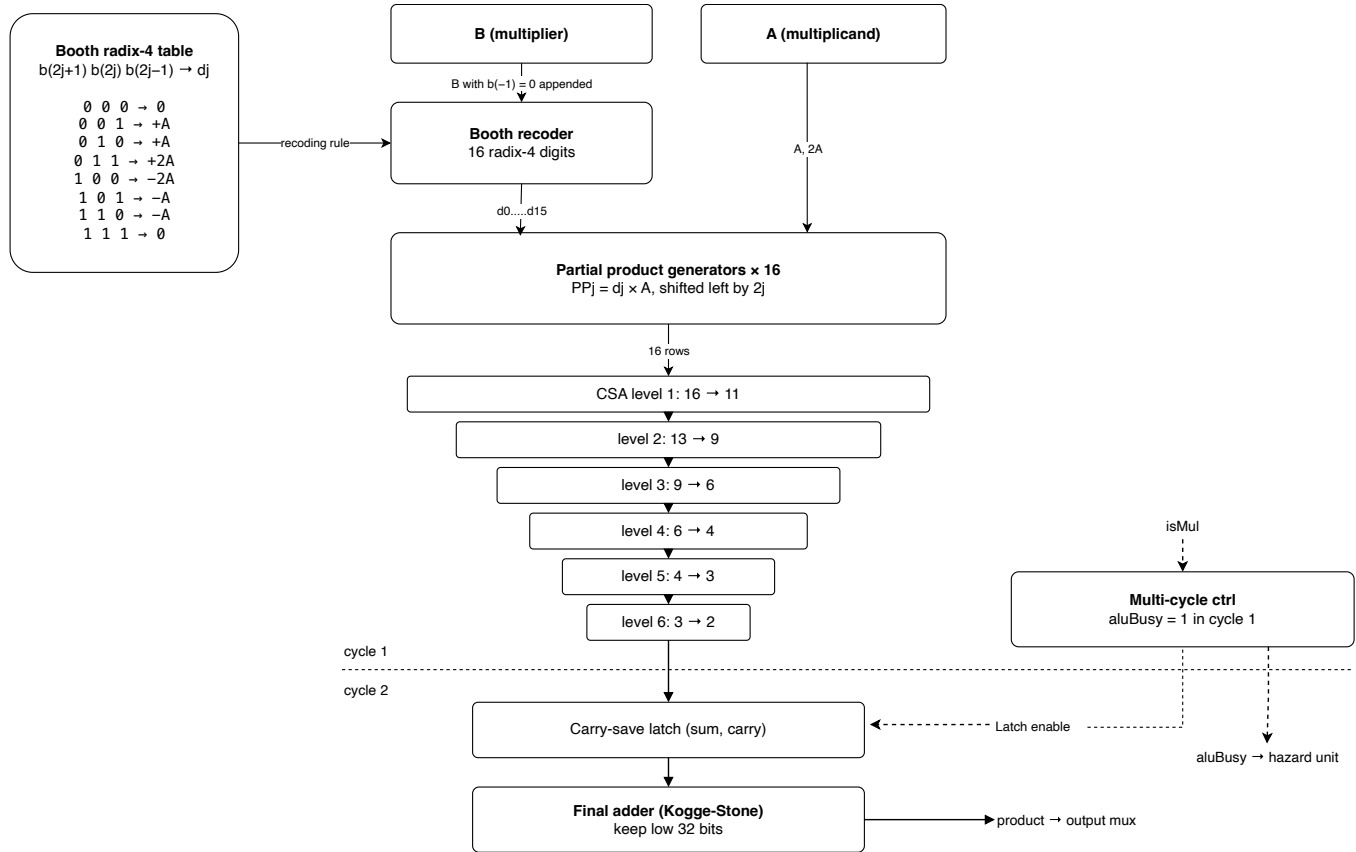
Kogge-Stone adder/subtractor



Multiplier

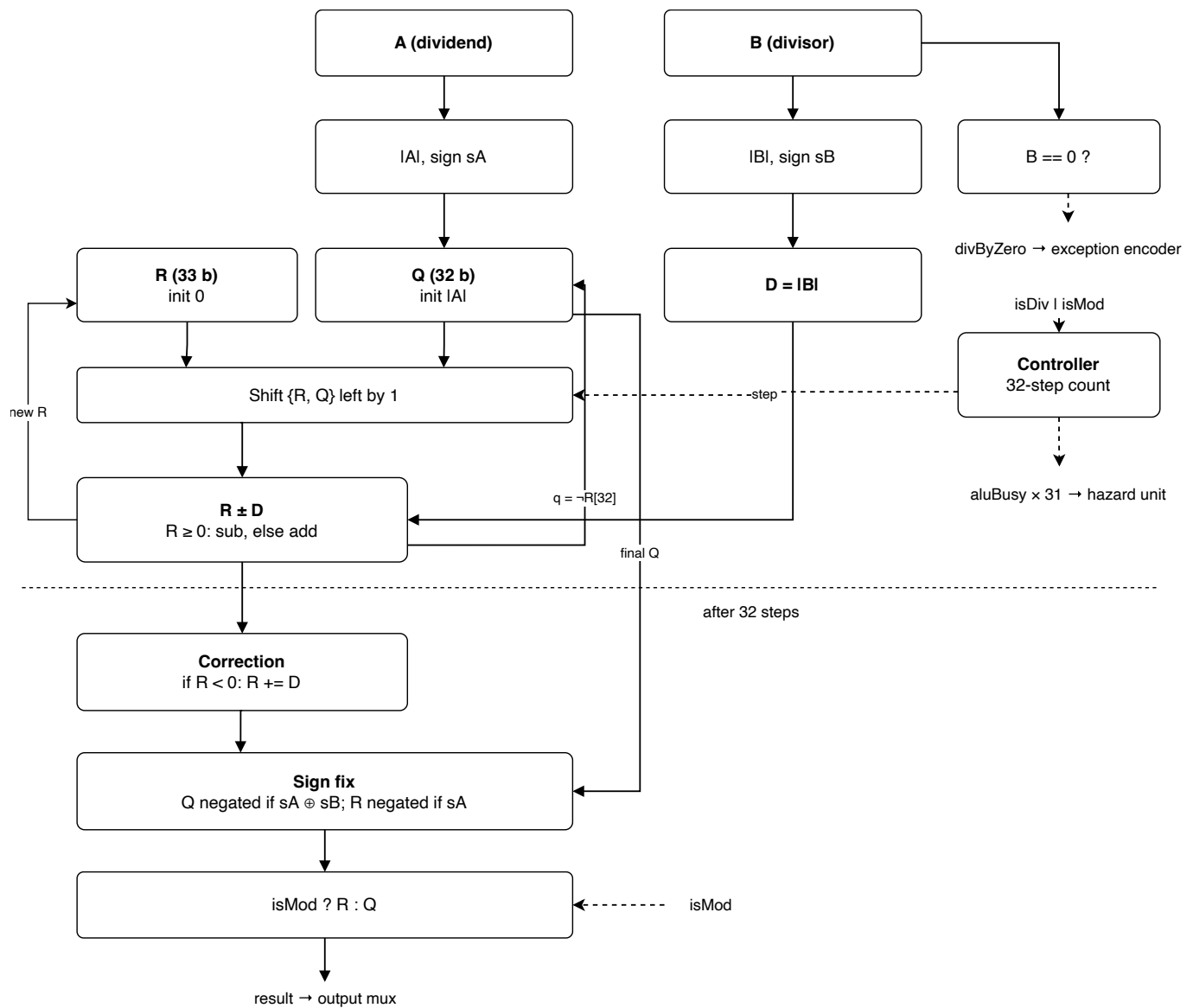
Implemented using radix-4 Booth encoding with a Dadda tree. Booth recoding halves the partial products ($32 \rightarrow 16$), and the Dadda tree reduces them to 2 rows in 6 levels using the minimum number of carry-save adders, giving the same speed as a Wallace tree with less hardware. It is split over 2 cycles with a

carry-save latch, and the final addition uses the Kogge-Stone adder.



Divider

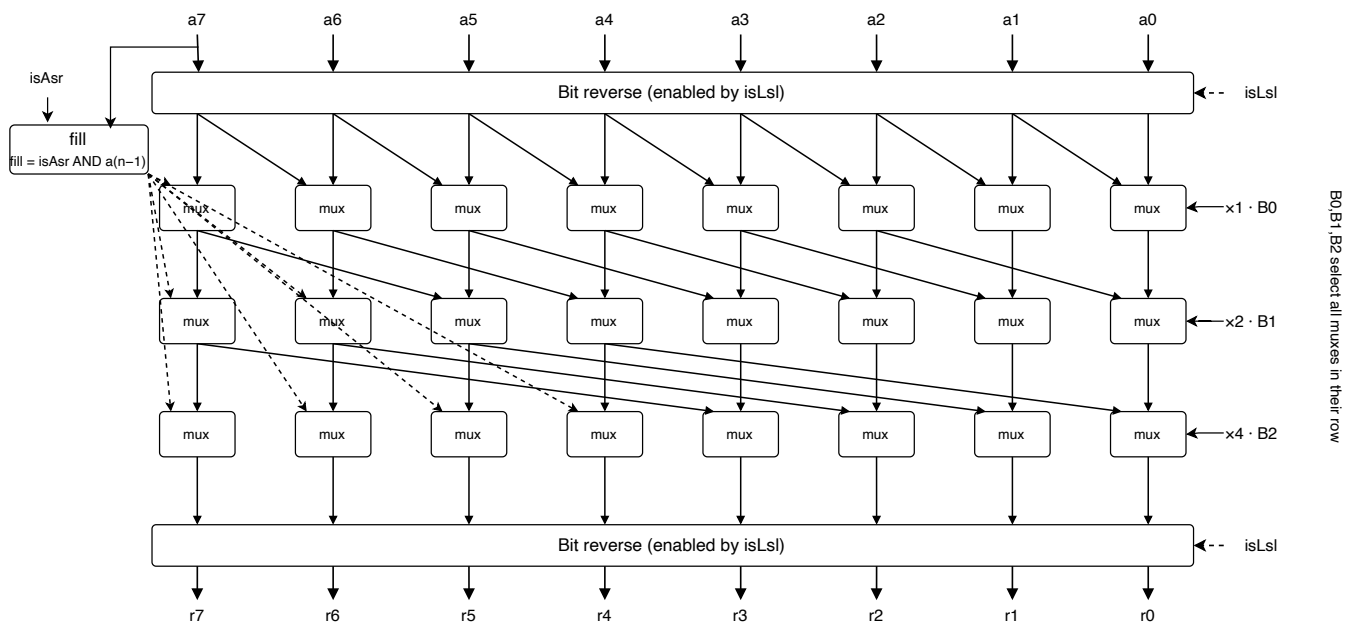
Implemented using the non-restoring division algorithm, shared by div and mod. Unlike restoring division, it never undoes a wrong subtraction; it corrects it with an addition in the next step, so each of the 32 steps needs only one add/subtract. It runs on magnitudes with a final sign fix, takes 32 cycles (aluBusy stalls the pipeline), and raises divByZero when $B = 0$.



Shifter

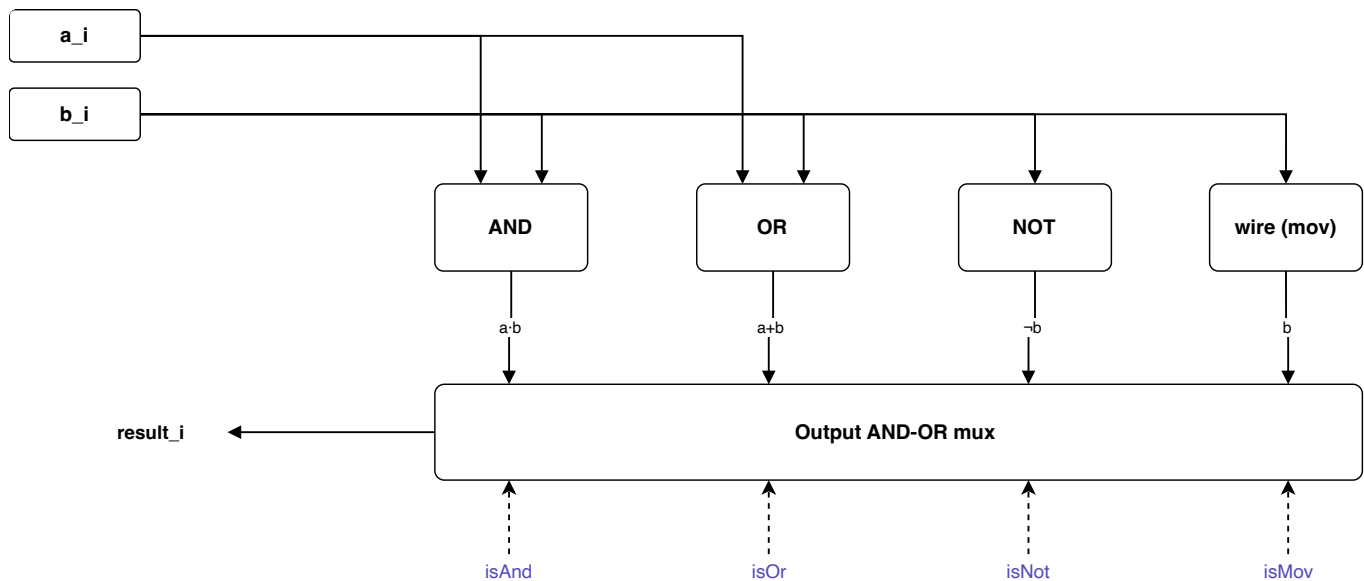
Implemented as a logarithmic barrel shifter, which shifts by any amount (0–31) in a single cycle using 5 layers of 2:1 muxes (shift by 1, 2, 4, 8, 16), each controlled by one bit of the shift amount. A single right-shifting structure handles all three shifts: asr fills vacated bits with the sign bit, and lsl reuses the same

hardware by reversing the bits before and after the shift.



Logical Unit + Mov

Implemented as a bitwise logic slice repeated 32 times. AND and OR take both operands, while NOT and mov use only B (not rd, rs2/imm, mov rd, rs2/imm). All four results are computed in parallel every cycle, and the output mux keeps only the one selected by $isAnd$, $isOr$, $isNot$ or $isMov$. This needs just one gate level, so it finishes in a single cycle.



One bit slice, repeated $\times 32$.